

MODULE 4 · WEEK 1

The Window Method for Time Series

Turn a time series into lag features so any model you already know can forecast it —
no special library required.

Lectures 40–42 · 45



This week

- 1 **EDA for time series** — grouping data and subplots
- 2 **The window method** — lag features unlock ML on time
- 3 **Splitting correctly** — time must flow forward
- 4 **Moving averages** — smoothing and momentum
- 5 **Interpretability** — which lags matter most



Why time series is different

- Time-indexed data is everywhere — prices, energy demand, traffic, sales, weather.
- Rows aren't independent: today depends on yesterday.
- Most intro courses skip it — but most of the data world runs on it.

Exploring time series: grouping & subplots

- Resample and **group** to see patterns at the right resolution (hourly, daily, monthly).
- Use **subplots** to compare several series or several windows side by side.
- Look for trend, seasonality, and gaps before you model.

```
fig, ax = plt.subplots(2, 1, figsize=(10, 6))  
df['value'].plot(ax=ax[0]); df['value'].rolling(24).mean().plot(ax=ax[1])
```

The window method (lag features)

- Key insight: build **lag features** — "what was the value 1, 6, 24 hours ago?"
- Now each row is a normal feature vector, and **any** model (Random Forest, boosting) can use it.
- No special time-series library needed — it's just feature engineering.

```
for lag in [1, 6, 24]:  
    df[f'lag_{lag}'] = df['value'].shift(lag)  
df = df.dropna()
```

Key idea Lag features turn 'predict the future' into ordinary supervised learning.

Split correctly — time flows forward

- **Never shuffle** a time series split — that leaks the future into the past.
- Train on earlier data, test on later data.
- Honest evaluation means the test set is always 'the future' the model hasn't seen.

```
split = int(len(df) * 0.8)
train, test = df.iloc[:split], df.iloc[split:]
```

Key idea Random shuffling on time series silently cheats — keep the timeline intact.



Moving averages & momentum

- A **moving average** smooths noise to reveal the underlying trend.
- Comparing a short vs. long average is the basis of **momentum** signals (e.g. trading).
- These are also useful engineered features for your model.



Which lags matter?

- Fit the model, then use permutation importance on the lag features.
- It tells you the memory of the system — e.g. the 24-hour lag dominates daily patterns.
- Interpretation closes the loop: xAI works on time series too.



Lab this week

- Build a feature matrix from a time series with the window method, split it correctly (no shuffle), and fit a Random Forest.
- Identify which lag features matter most.

In the labs repo: `Module4/Week1_WindowMethod/window_method.py`



Key takeaways

- Time series rows are dependent — today builds on yesterday.
- The window method creates lag features so any ML model can forecast.
- Split by time, never by shuffle — the test set is the future.
- Moving averages smooth trend and power momentum signals.
- Permutation importance reveals which lags carry the signal.